Topic 2, Lesson 2

Amazon EC2 and Compute Services



Outline

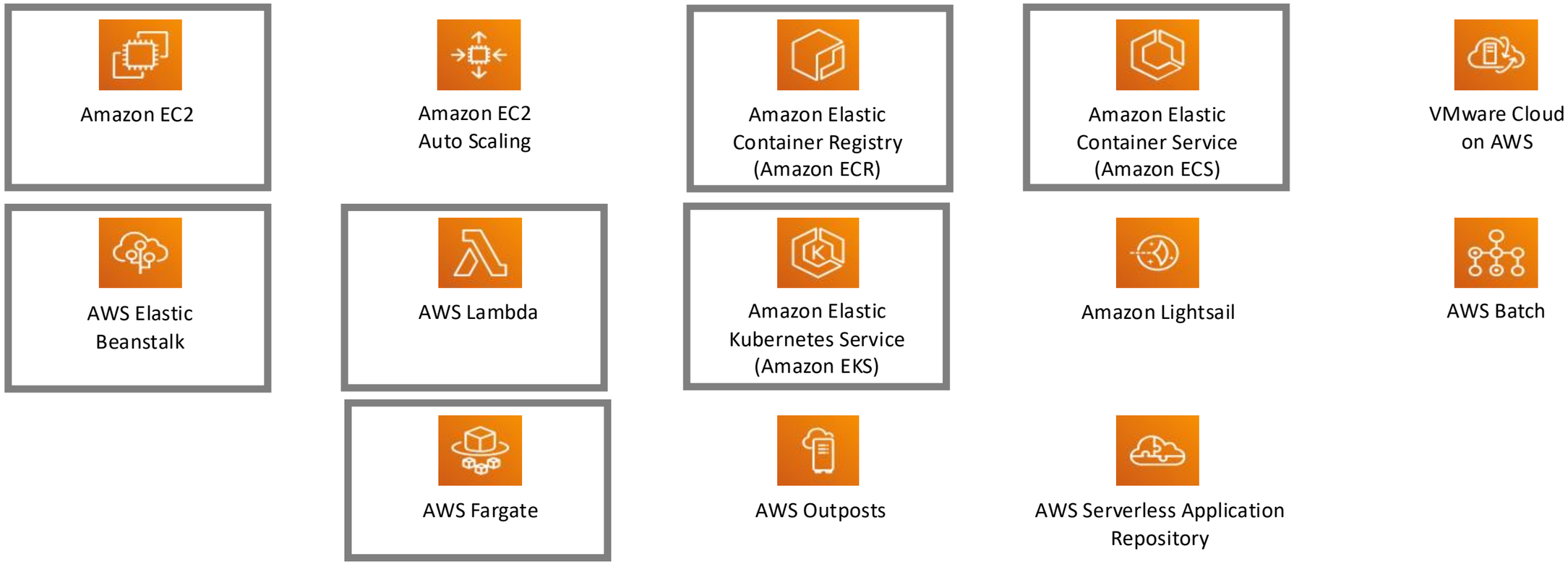
- AWS compute evolution.
- EC2
- Launching and managing EC2 instances
- Accessing EC2 instances securely
- Elastic Load Balancing (ELB)
- Auto Scaling Groups (ASG)

Types of Compute services

Services	Key Concepts	Characteristics	Ease of Use
• Amazon EC2	• Infrastructure as a service (IaaS) • Instance-based • **Virtual machines**	• Provision virtual machines that you can manage as you choose	A familiar concept to many IT professionals.
• AWS Lambda	• **Serverless** computing • Function-based • Low-cost	• Write and deploy code that runs on a schedule or that can be triggered by events • Use when possible (architect for the cloud)	A relatively new concept for many IT staff members, but easy to use after you learn how.
• Amazon ECS • Amazon EKS • AWS Fargate • Amazon ECR	• **Container-based** computing • Instance-based	• Spin up and run jobs more quickly	AWS Fargate reduces administrative overhead, but you can use options that give you more control.
• AWS Elastic Beanstalk	• Platform as a service (PaaS) • For **web applications**	• Focus on your code (building your application) • Can easily tie into other services—databases, Domain Name System (DNS), etc.	Fast and easy to get started.

AWS's Compute Evolution

Amazon Web Services (AWS) offers many compute services.



EC2: AWS's Compute Foundation

What is Amazon EC2?

- Amazon Elastic Compute Cloud (EC2) provides resizable virtual machine, known as "instances," with attachable storage, networking functionality, and identity management.

- These resources are provisioned through **Application Programming Interface (API) calls**, the **AWS Console**, or **CLI tools**, giving you full programmatic control over your infrastructure.

Common uses for EC2 instances include, but are not limited to:

- Application servers
- Web servers
- Database servers
- Game servers
- Mail servers
- Media servers
- Catalog servers
- File servers
- Computing servers
- Proxy servers
- Hosting websites
- Big data processing
- Machine learning workloads

Why was EC2 created?

EC2 fundamentally changed how organizations approached compute resources. It replaced the capital expenditure and slow procurement cycles of hardware with on-demand compute capacity that scales with your needs. This shift enabled rapid experimentation and eliminated the risk of over-provisioning or under-provisioning infrastructure. It means it is-

- Scalable
- Pay-as-you-go
- Multiple OS choices
- Secure access Easy automation.

When NOT to use EC2

Consider using containerization (ECS/EKS) for microservices architectures or serverless options (Lambda) for event-driven workloads. These alternatives often provide better cost efficiency and operational simplicity when they fit your use case.

When to use EC2

- Need full operating system control
- Licensing or compliance constraints

Key Terminology

- Instance,
- AMI,
- EBS
- Security Group
- ENI
- Load Balancer
- Auto Scaling Group



AWS Global Infrastructure: Where Your Machines Run

Region

Geographic areas such as ap-southeast-1 (Singapore) or us-east-1 (Virginia). Each Region is completely independent to ensure data sovereignty and fault isolation.

Availability Zone

Isolated data centers within a Region, connected by low-latency links. Design multi-AZ deployments to achieve high availability and fault tolerance.

Data Center

Physical infrastructure organized into racks. Used by placement strategies to control the distribution of Instances.

Physical Placement Control: Placement Groups

Cluster Placement

Groups instances in close proximity within a single Availability Zone to achieve the lowest possible network latency. Ideal for High-Performance Computing (HPC) applications with tight node-to-node communication.

Use cases: High-Performance Computing (HPC), Machine Learning training with distributed workloads

Spread Placement

Places each instance on separate underlying hardware racks to minimize correlated failures.

Use cases: Critical applications requiring maximum isolation from hardware failures

Partition Placement

Divides instances into partitions, with each partition on separate sets of racks. Each partition has its own network and power.

Use cases: Large distributed systems like Kafka, HDFS, and Cassandra clusters

The **Nitro Platform** represents AWS's modern virtualization technology, delivering near bare-metal performance while maintaining strong security isolation between instances.



Instance Families: Choosing the Right Hardware

General Purpose (M/T)

Balanced compute, memory, and network resources. Perfect for web servers, application tiers, and development environments where no single resource predominates.

Compute Optimized (C)

High-performance processors for compute-intensive tasks. Best for batch processing, video encoding, scientific modeling, and dedicated gaming servers.

Memory Optimized (R/X/U)

Large memory capacity for in-memory databases, real-time big data analytics, and high-performance caching tiers like Redis or Memcached.

Other Instance Families

Storage Optimized (I/D/H)

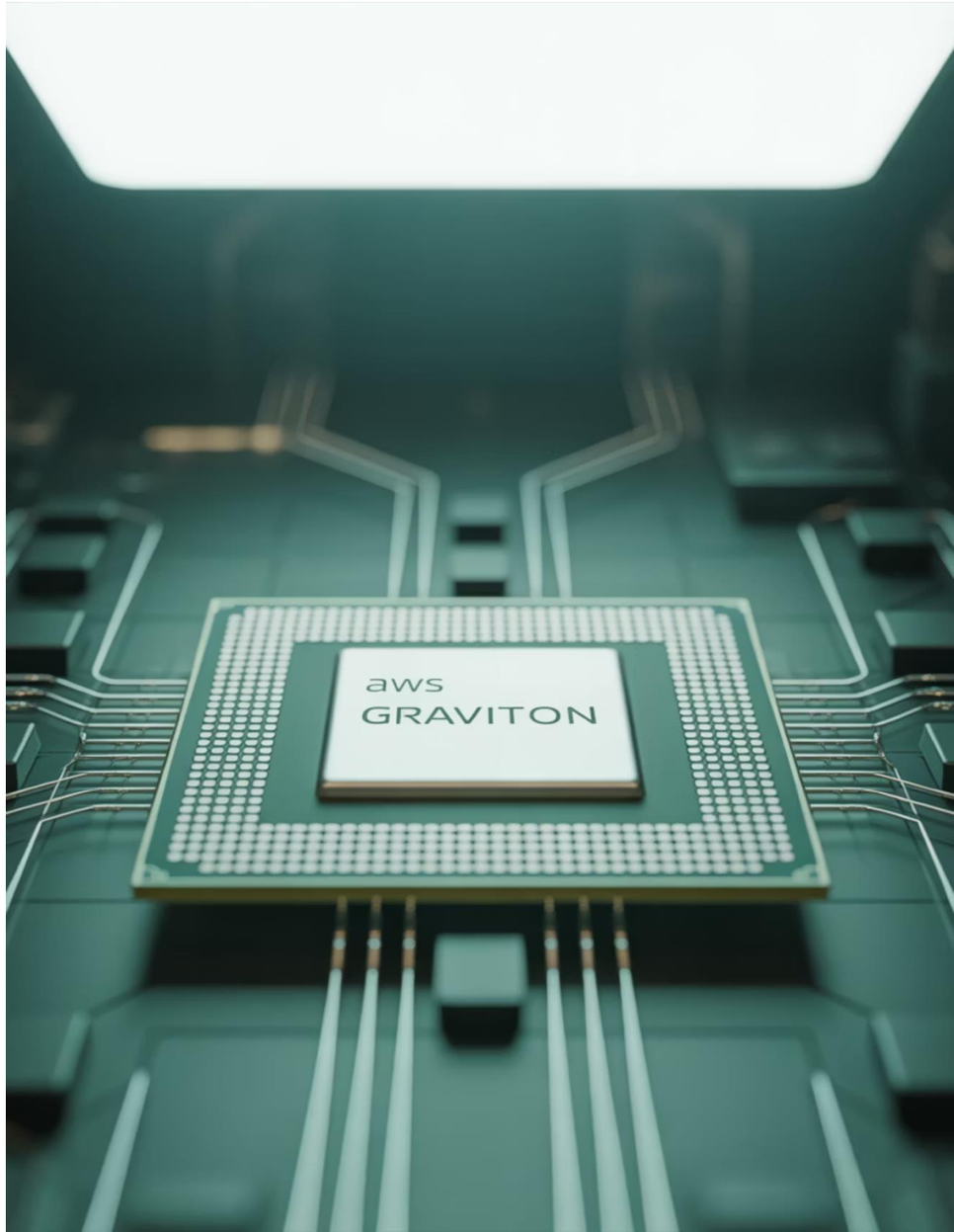
Utilize high-speed local NVMe SSD storage with high IOPS, suitable for data warehouses, distributed file systems, and NoSQL databases requiring high sequential read/write access.

Accelerated Computing (P/G/Inf)

Use GPUs and machine learning accelerators for deep learning training, inference, graphics rendering, and parallel floating-point calculations.

Burstable Performance (T3/T4g)

Cost-effective instances with a baseline CPU performance and the ability to burst above the baseline using CPU credits. Note the potential for performance throttling under sustained high load.



AWS Graviton Processors (ARM): Instance types with a "g" suffix (e.g., t4g, m6g) use Graviton processors based on ARM architecture, offering up to 40% better price-performance. Ensure your application is compiled for ARM64 architecture.

AWS pricing model

Three fundamental drivers of cost with AWS

Compute

- Charged per hour/second*
- Varies by instance type

*Linux only

Storage

- Charged typically per GB

Data transfer

- Outbound is aggregated and charged
- Inbound has no charge (with some exceptions)
- Charged typically per GB



Overview of EC2 Pricing Models

1

On-Demand Instances

Pay by the second with no long-term commitment. Highest flexibility, highest cost. Ideal for unpredictable workloads and short-term testing.

2

Savings Plans

Commit to a consistent dollar amount per hour for 1-3 years in exchange for significant discounts (up to 72% off On-Demand).

3

Reserved Instances

Commit to specific instance attributes (family, region, OS) for stable, predictable workloads. Regional Reserved Instances also reserve capacity.

4

Spot Instances

Leverage unused AWS capacity at up to 90% discount. Instances can be interrupted with 2 minutes notice. Perfect for fault-tolerant batch jobs.

Significant Cost Savings with Risk: Spot Instances

How Spot Instances Work

Spot Instances allow you to purchase unused EC2 capacity at significant discounts, sometimes up to 90% off On-Demand prices. However, AWS can reclaim these instances with a 2-minute warning when capacity is needed for On-Demand or Reserved Instance customers.



Ideal Use Cases

Big data analytics, containerized workloads, CI/CD systems, batch processing, and rendering farms all benefit from the substantial cost savings of Spot.

Handling Interruptions: Monitor the instance metadata service to receive termination notices and implement flexible shutdown procedures to save work before termination.

Flexible Commitment Options: Savings Plans

Compute Savings Plans

The most flexible option - commit to a fixed amount per hour and receive discounts on EC2 (any instance family, any region), AWS Fargate, and AWS Lambda. Automatically applies to your usage regardless of instance family, size, OS, tenancy, or region.

Discount: Up to 66% compared to On-Demand prices

EC2 Instance Savings Plans

Deeper discounts but less flexible - commit to a specific instance family in a given region. Flexible on size, OS, and tenancy within that same instance family.

Discount: Up to 72% compared to On-Demand prices

Example: M5 instance family in us-east-1, any size (m5.large, m5.xlarge, etc.)

Decision: Use Savings Plans for stable computing needs, Spot Instances for flexible excess capacity, and reserve On-Demand Capacity Reservations for critical deployment periods.

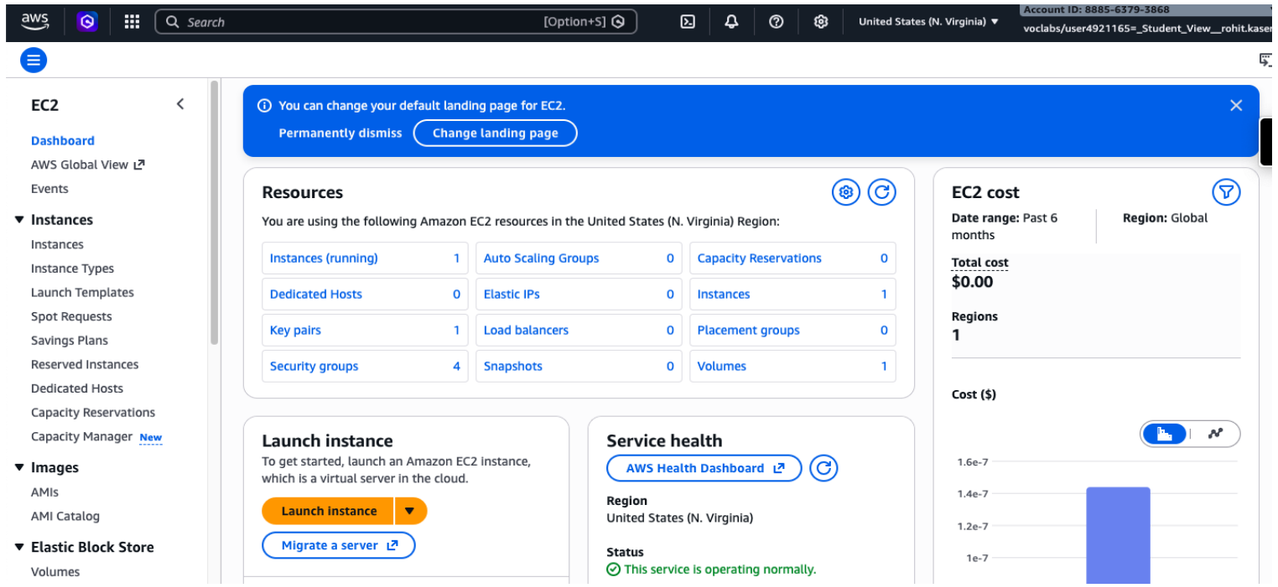
Launching an Amazon EC2 instance

Components required before EC2 Launch

- AMI (Amazon Machine Image) → OS template
- Instance Type → CPU / RAM size
- Key Pair → SSH / RDP access
- Security Group → firewall rules
- VPC + Subnet → networking
- EBS Volume → storage
- IAM Role → permissions (optional)

EC2 Launching Flow:

AMI → Instance Type → Network → Security Group → Key Pair → Launch



Steps to launch using AWS console:

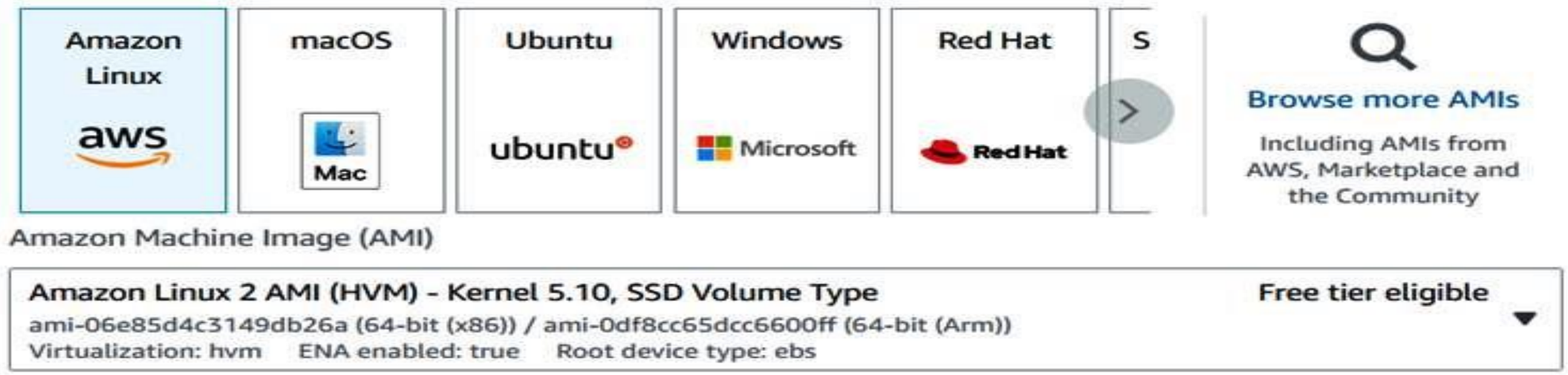
- Open AWS Console → EC2 Dashboard
- Click Launch Instance
- Enter instance name
- Choose AMI
- Select Instance Type
- Create/select Key Pair
- Configure Security Group
- Add Storage
- Click Launch Instance

Lunch EC2 using AWS console

- Note: Before launch new instance verify that your EC2 console is currently managing resources of the particular user region. Such as N. Virginia (us-east-1) region.
- Verify this by looking at the drop down menu at the top of the screen, to the left of your username.
- If it does not already indicate N. Virginia, choose the N. Virginia region from the region menu before proceeding to the next step.

1. Select AMI:

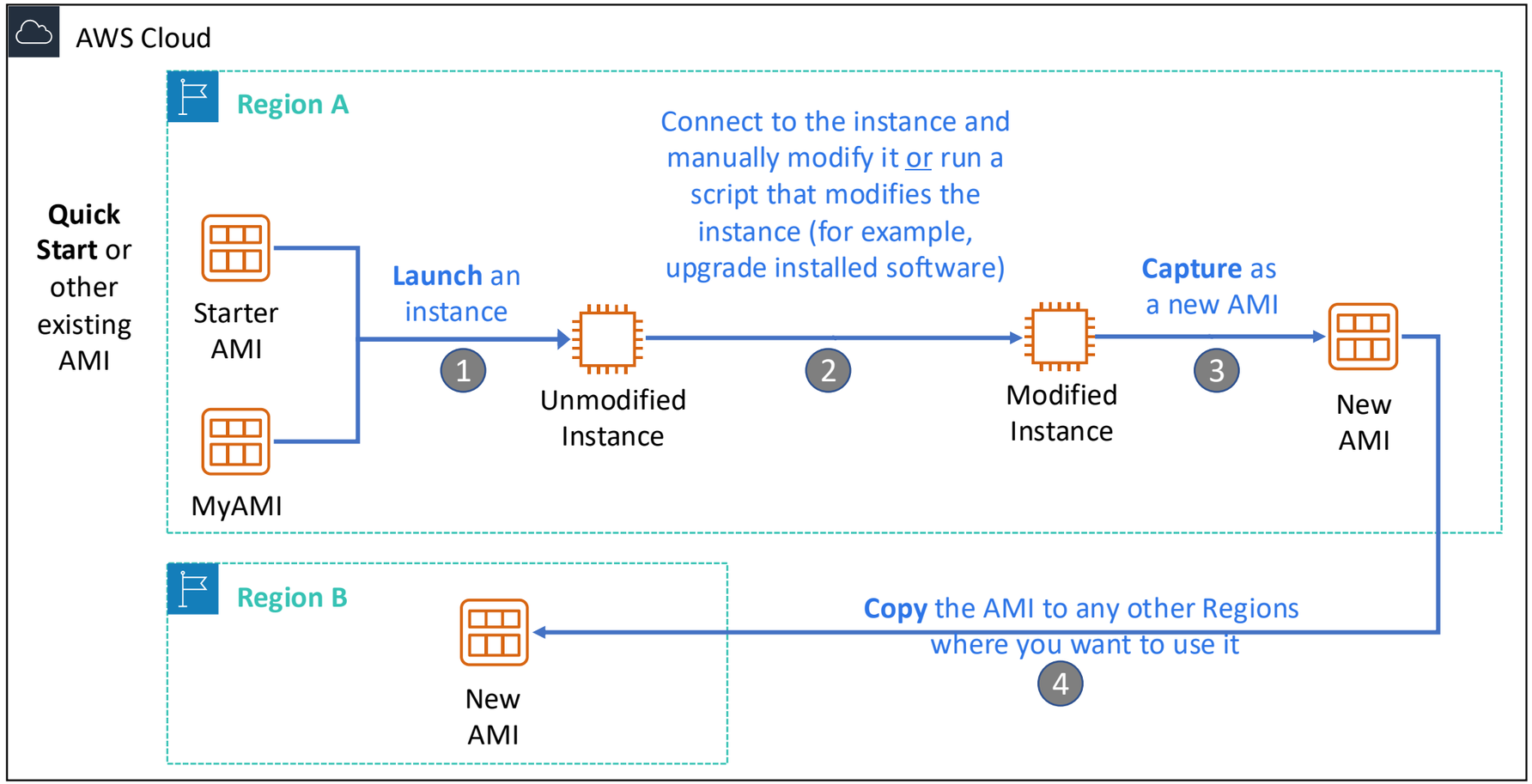
- Select the software of the EC2 instance: Think of it just like selecting software (OS and other applications) when purchasing a physical desktop or laptop PC.
- In AWS, the software image for an EC2 instance is called an Amazon Machine Image (AMI), which is a template that is used to launch an EC2 instance.
- Amazon provides AMIs in Windows, Linux, and other operating systems, customized with some other software pre-installed:



Creating a new AMI in AWS

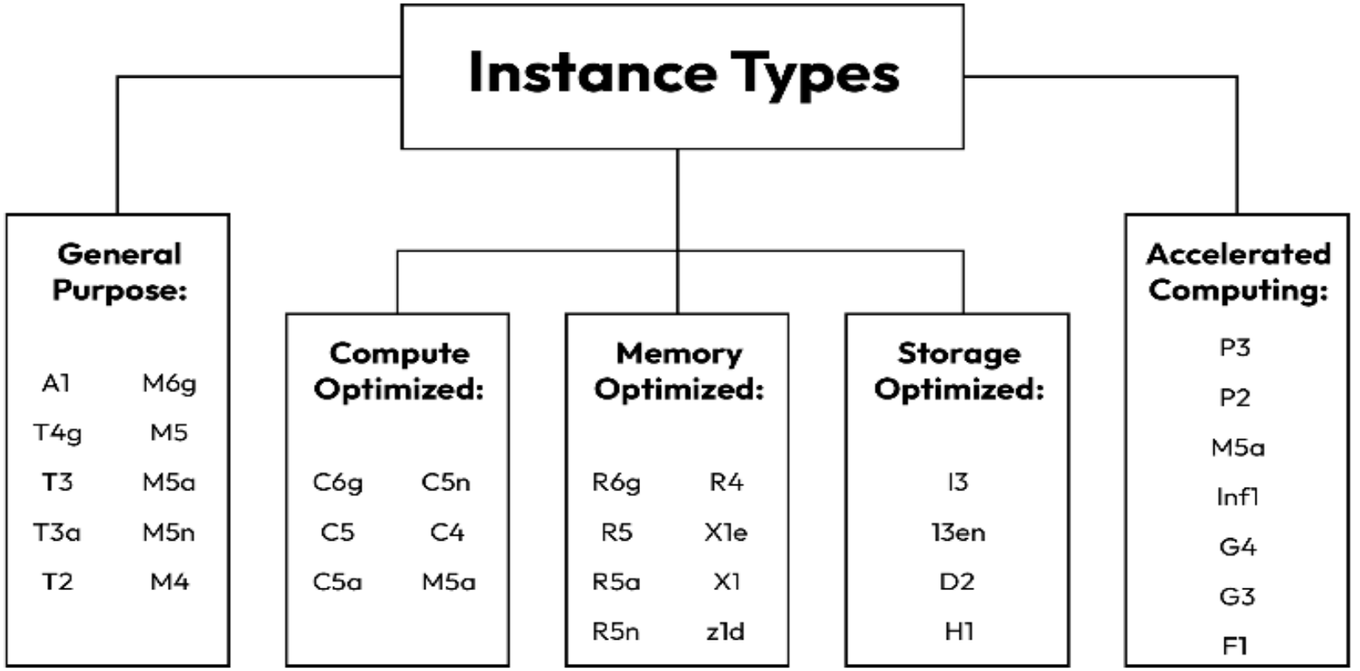
AMI details

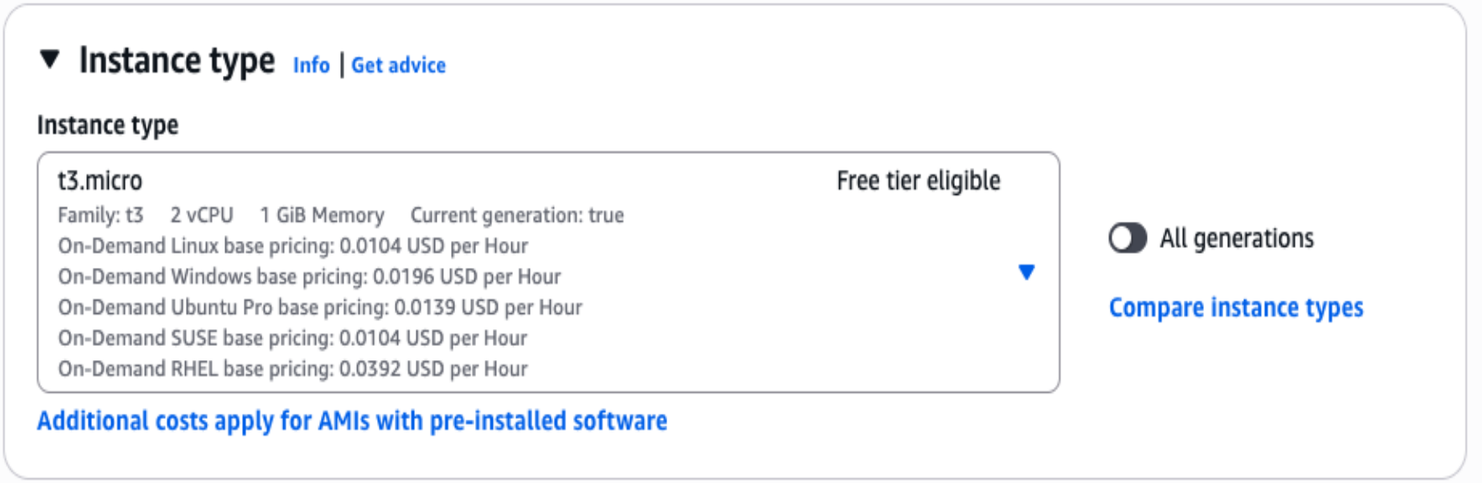
(Optional) Import a virtual machine



2. Select an Instance type

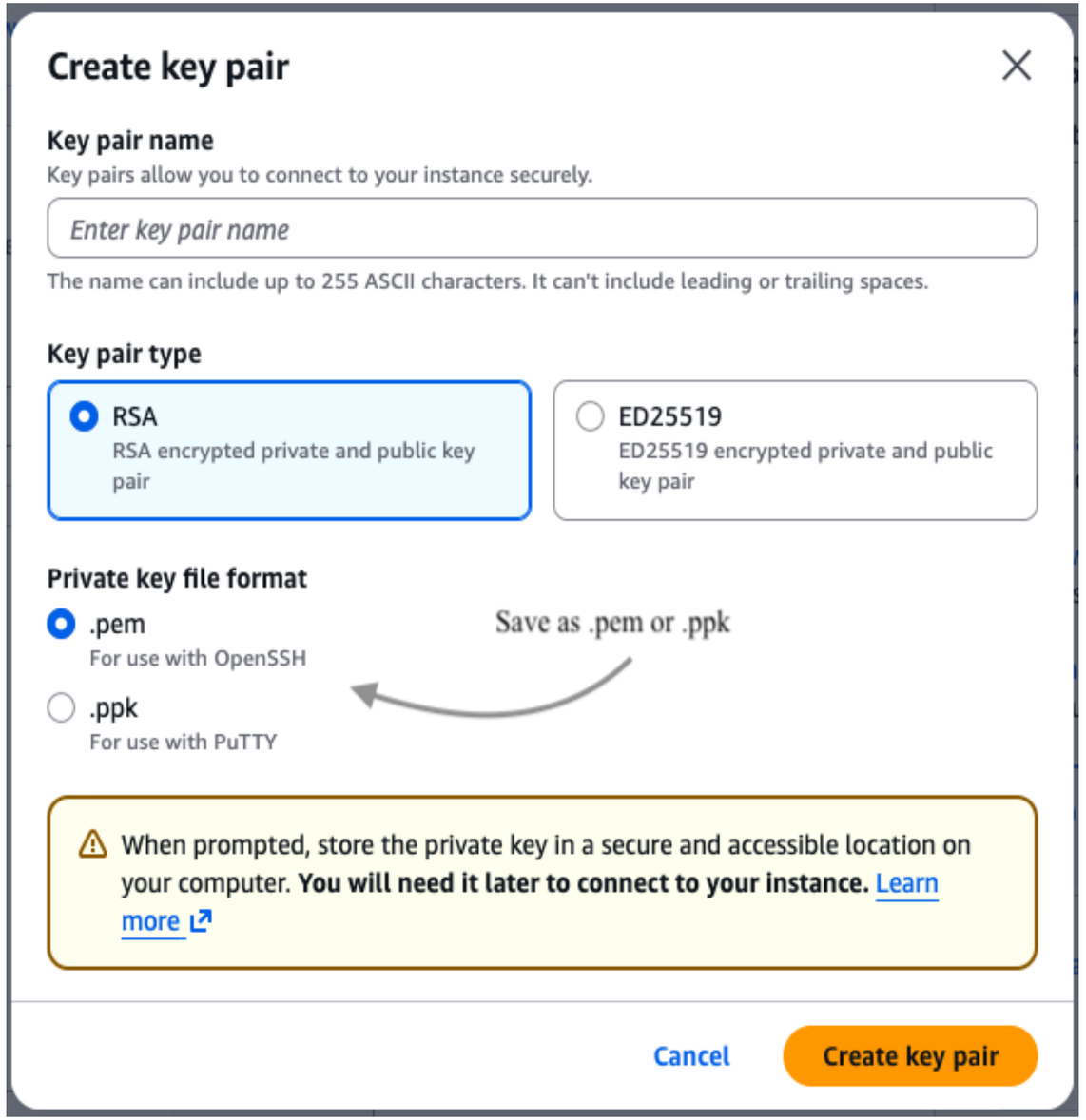
- Consider your use case
  - How will the EC2 instance you create be used?
- The **instance type** that you choose determines –
  - Memory (RAM)
  - Processing power (CPU)
  - Disk space and disk type (Storage)
  - Network performance
- Instance type categories –
  - General purpose
  - Compute optimized
  - Memory optimized
  - Storage optimized
  - Accelerated computing
- Instance types offer *family*, *generation*, and *size*





3. Key Pair Login

- Amazon EC2 uses public–key cryptography to encrypt and decrypt login information.
- To ensure you will be able to log in to the guest OS of the instance you create, you identify an existing key pair or create a new key pair when launching the instance.
- Amazon EC2 then installs the key on the guest OS when the instance is launched.
- That way, when you attempt to login to the instance and you provide the private key, you will be authorized to connect to the instance.



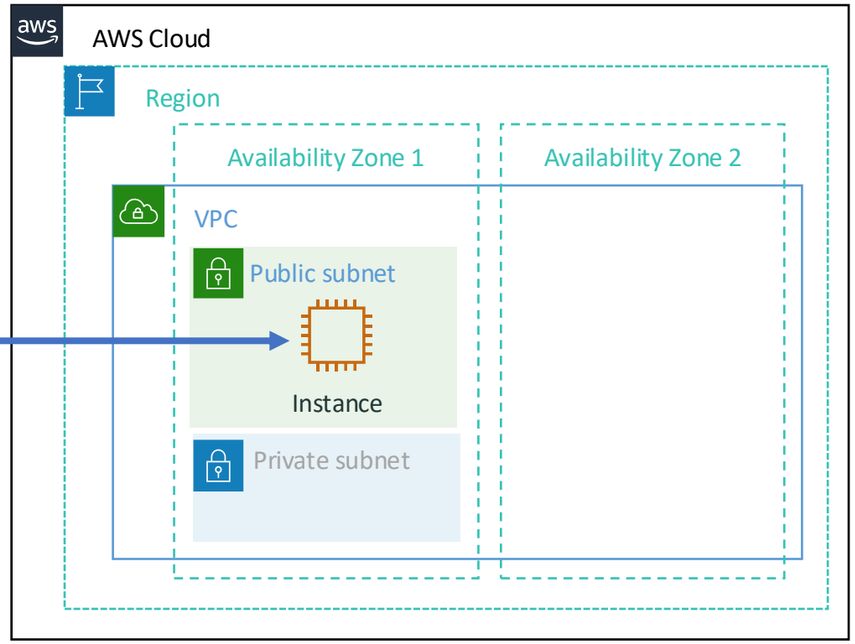
4. Specify network settings

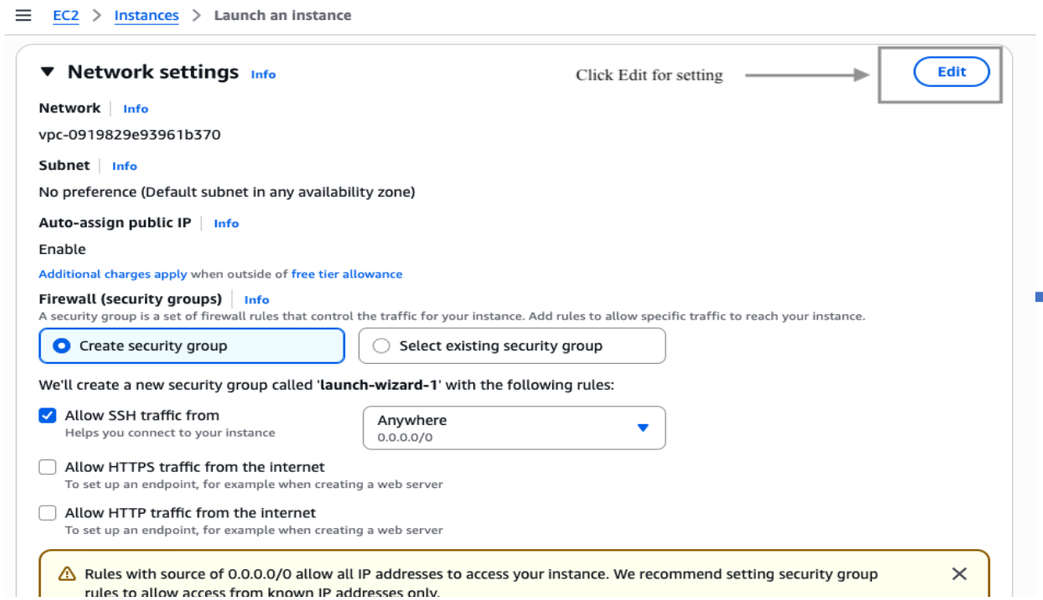
This is like subscribing to an Internet Service Provider (ISP) for our home PC to connect to a network and the internet.

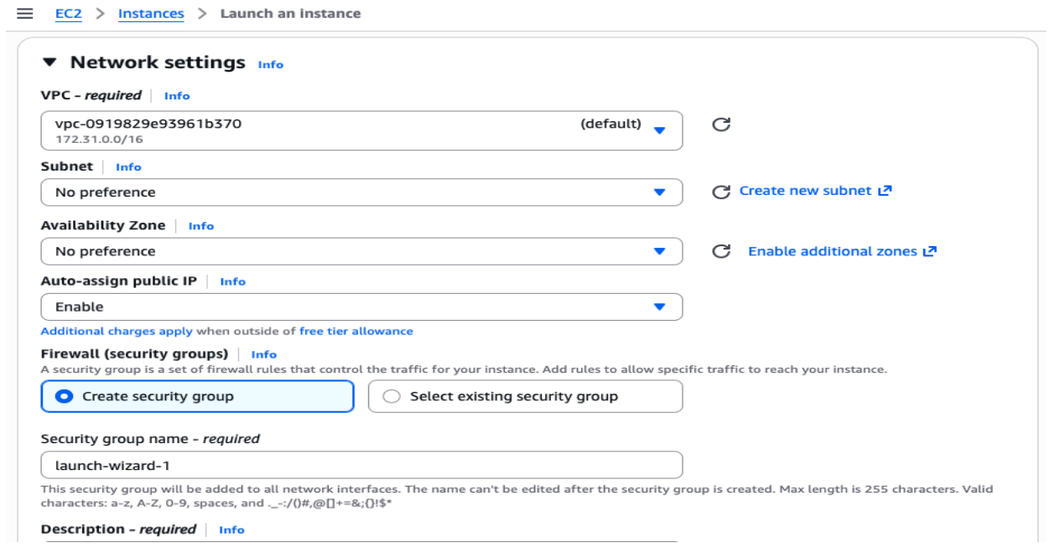
In the AWS cloud, the basic network unit is called a Virtual Private Cloud (VPC), and Amazon has provided a default VPC and subnets in each region.

- Where should the instance be deployed?
- Identify the VPC and optionally the subnet
- Should a public IP address be automatically assigned?
- To make it internet-accessible

Example: specify to deploy the instance here





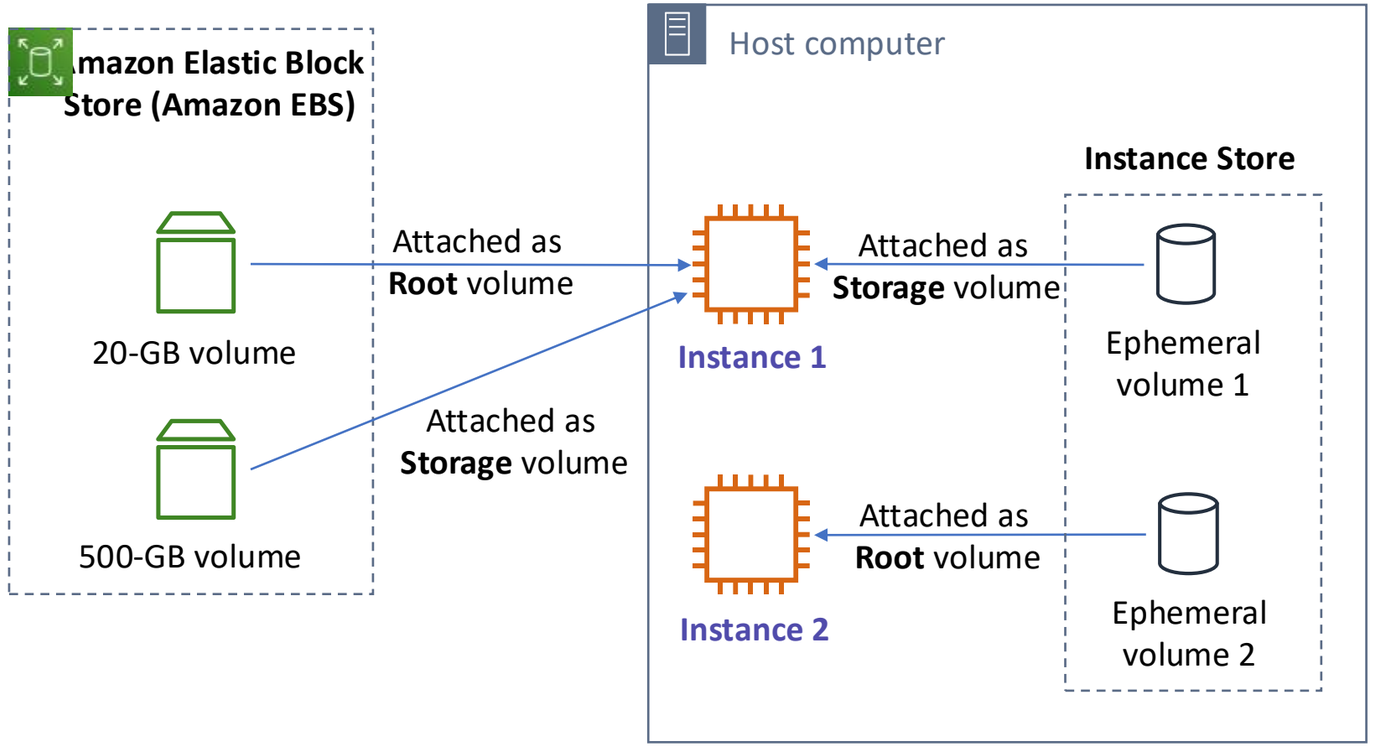


5. Configure Storage (Amazon EBS – Elastic Block Store) Optional

EBS is basically the cloud hard disk attached to an EC2 virtual machine. It stores the OS, applications, and data persistently.
Configure Storage allows you to define the disk size, type, performance, and persistence for your EC2 server.

Main Storage Options

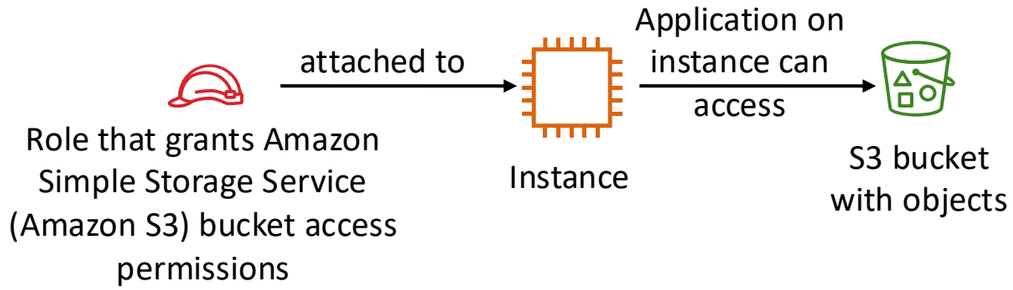
- Root Volume (EBS): OS disk for Windows/Linux
- Additional EBS Volumes: Extra storage for data, logs, backups
- Volume Types:
  - gp3 SSD → best default, balanced cost + speed
  - io2 SSD → high IOPS for databases
  - st1 HDD → throughput workloads
  - sc1 HDD → low-cost archive
- Instance Store: Temporary ultra-fast local storage
- Snapshots & Encryption: Backup and secure data with KMS



Important: HDD volumes cannot be used as boot volumes.

6. Attach IAM role (optional)

- Will software on the EC2 instance need to interact with other AWS services?
  - If yes, attach an appropriate **IAM Role**.
- An AWS Identity and Access Management (IAM) role that is attached to an EC2 instance is kept in an **instance profile**.
- You are *not* restricted to attaching a role only at instance launch.
  - You can also attach a role to an instance that already exists.
- An IAM role can be created and assigned with permissions to access other AWS resources, such as reading an **Amazon Simple Storage Service** (**Amazon S3**) bucket.



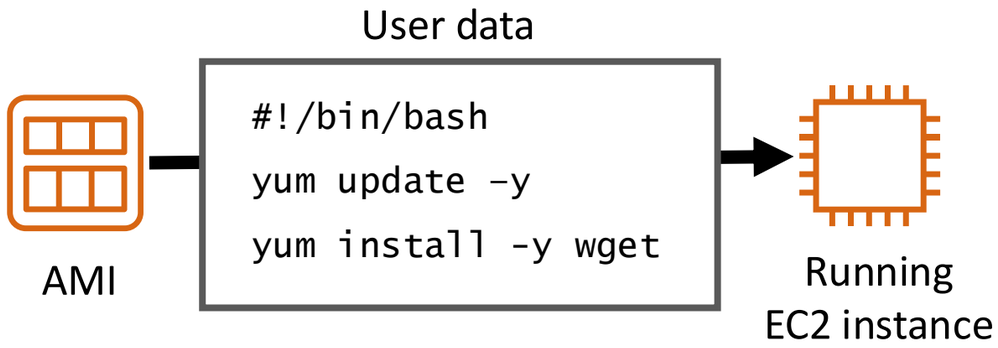
Attaching an IAM role to an EC2 instance



7. Automating Initial Startup: User Data (optional)

What is User Data?

User data consists of scripts or `cloud-init` directives that automatically run when an instance starts for the first time. This enables automatic configuration without needing to bake everything into your AMI.



- Optionally specify a user data script at instance launch
- Use **user data** scripts to customize the runtime environment of your instance
  - Script runs the first time the instance starts
- Can be used strategically
  - For example, reduce the number of custom AMIs that you build and maintain

```
#!/bin/bash
dnf install -y httpd
Systemctl enable httpd
systemctl start httpd
echo '<html><h1>Hello From Your Web
Server!</h1></html>' > /var/www/html/index.html
```

Other optional steps to launch EC2 using Console

- Optionally assign a tag to the EC2 instance: A tag is a label that we can assign to an AWS resource, and it consists of a key and an optional value.
- Instance auto-recovery
- Hostname type
- Instance auto-recovery
- Termination protection
- Stop protection
- Metadata accessible

Note: User data scripts run as root and execute only on the first boot by default. Use `cloud-init` directives for more control.

Launching EC2 instances using CloudShell

This is a browser-based terminal inside AWS Console.
Why it is needed ?

- ✓ No CLI installation required
- ✓ Already authenticated
- ✓ Uses current console permissions
- ✓ Great for quick admin tasks

- Launch EC2 instances using the command line in CloudShell, which is a browser-based, pre-authenticated shell that you can launch directly from the AWS Management Console.

Steps:

- From the AWS console, launch CloudShell by clicking the CloudShell sign.

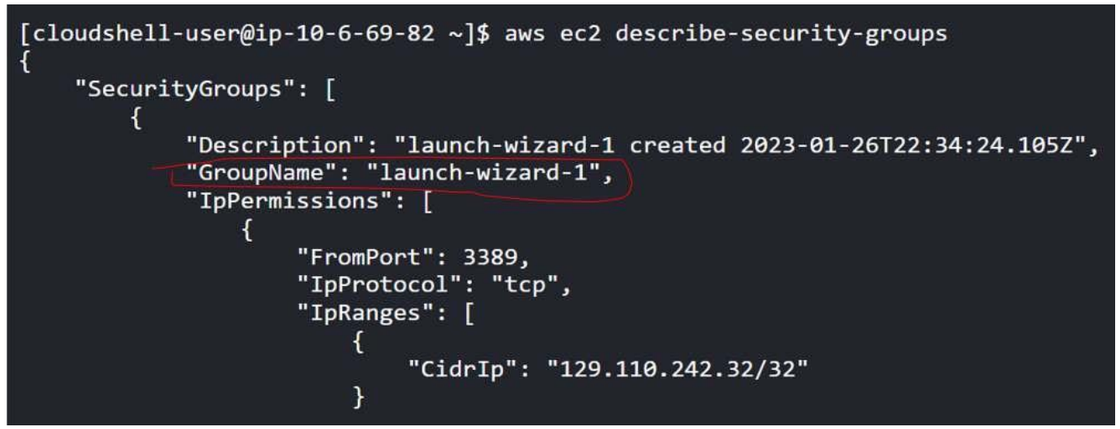


- Find the AWS AMI image ID in the particular region, with the following CloudShell command:

```
[cloudshell-user]$ aws ec2 describe-images --region
us-west-2
```



- Find the security group name using below command given in figure.



- Find the key pair using below command given in figure.

```
[cloudshell-user@ip-10-6-69-82 ~]$ aws ec2 describe-key-pairs
{
    "KeyPairs": [
        {
            "KeyPairId": "key-092be0798f7c7883b",
            "KeyFingerprint": "c7:4d:81:09:8c:3d:cb:db:b0:47:ee:4e:21:dd:53:cd:54:5a:65:72",
            "KeyName": "mywestkp",
            "KeyType": "rsa",
            "Tags": [],
            "CreateTime": "2023-01-26T22:35:40+00:00"
        }
    ]
}
```

- Create an EC2 instance in the particular region, using the aws ec2 run-instances command, with the following configurations we obtained from the previous steps.

```
aws ec2 run-instances --image-id ami-0ef0b498cd3fe129c
--count 1 --instance-type t2.micro --key-name mywestkp
--security-groups launch-wizard-1 --region us-west-2
```

```
[cloudshell-user@ip-10-6-69-82 ~]$ aws ec2 run-instances  --image-id ami-0ef0b498cd3fe129c  --count 1  --instance-type t2.micro  --
key-name mywestkp --security-groups launch-wizard-1  --region us-west-2
{
    "Groups": [],
    "Instances": [
        {
            "AmiLaunchIndex": 0,
            "ImageId": "ami-0ef0b498cd3fe129c",
            "InstanceId": "i-00398bfb35733237a",
            "InstanceType": "t2.micro",
            "PublicDnsName": "",
```

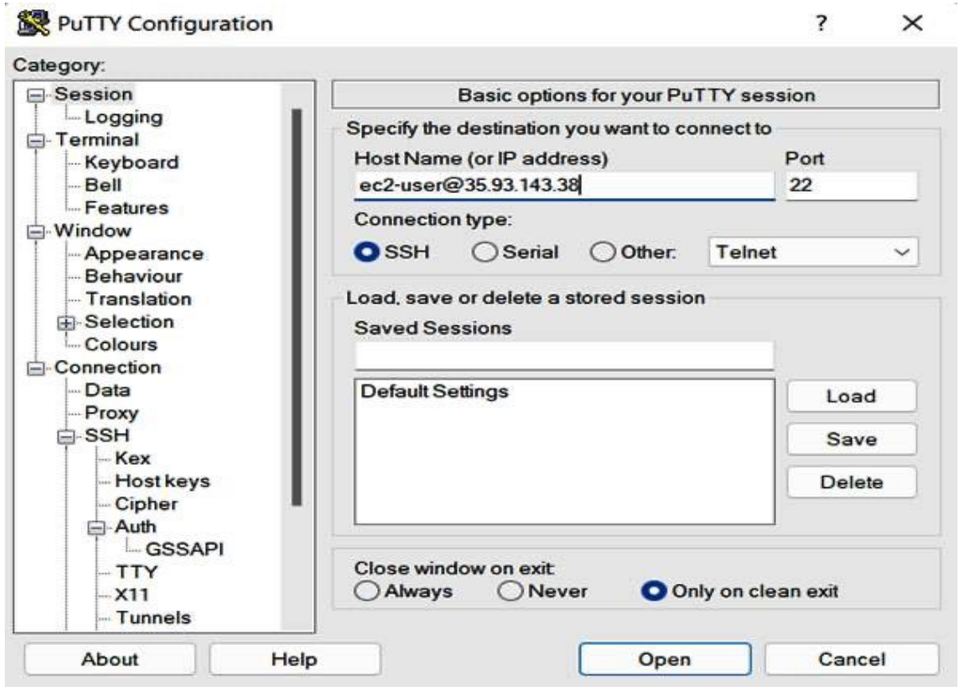
- Examine the details of the instance with public IP from its InstanceId value. instance has a public IP address of 35.93.143.38

```
[cloudshell-user@ip-10-6-69-82 ~]$ aws ec2 describe-instances --instance-ids i-00398bfb35733237a
{
    "Reservations": [
        {
            "Groups": [],
            "Instances": [
                {
                    "AmiLaunchIndex": 0,
                    "ImageId": "ami-0ef0b498cd3fe129c",
                    "InstanceId": "i-00398bfb35733237a",
                    "InstanceType": "t2.micro",
                    "KeyName": "mywestkp",
                    "LaunchTime": "2023-01-28T14:51:40+00:00",
                    "Monitoring": {
                        "State": "disabled"
                    "LaunchTime": "2023-01-28T14:51:40+00:00",
                    "Monitoring": {
                        "State": "disabled"
                    },
                    "Placement": {
                        "AvailabilityZone": "us-west-2a",
                        "GroupName": "",
                        "Tenancy": "default"
                    },
                    "PrivateDnsName": "ip-172-31-23-118.us-west-2.compute.internal",
                    "PrivateIpAddress": "172.31.23.118",
                    "ProductCodes": [],
                    "PublicDnsName": "ec2-35-93-143-38.us-west-2.compute.amazonaws.com",
                    "PublicIpAddress": "35.93.143.38",
```

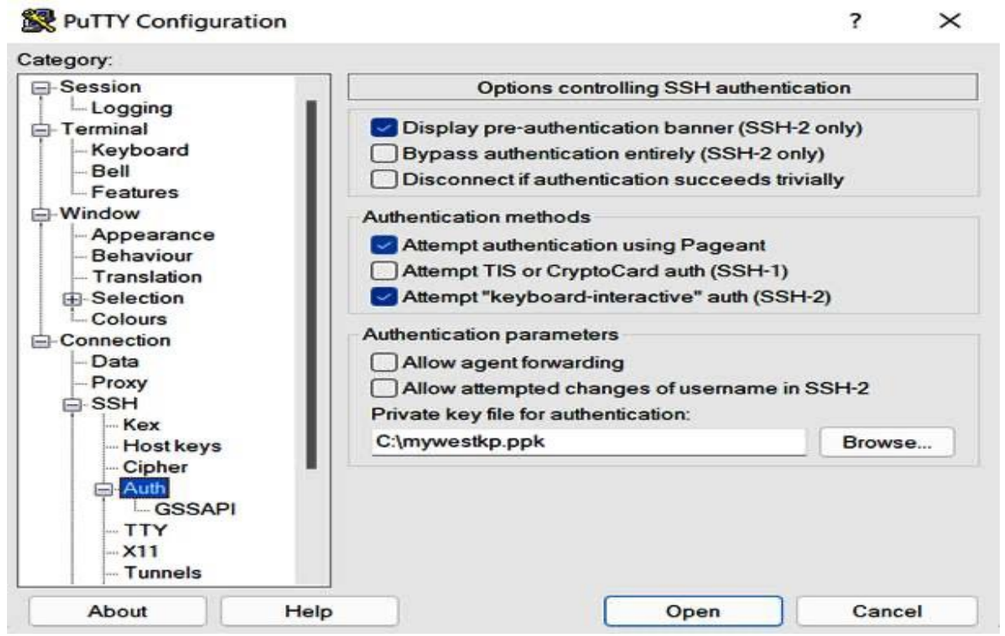
➢ Logging in to the EC2 instances

- Use SSH to access the Linux EC2 instance.
- **PuTTY** is a free and open source terminal emulator, serial console, and network file transfer application.
- Download PuTTY and use it to connect to the Linux VM in the AWS cloud.

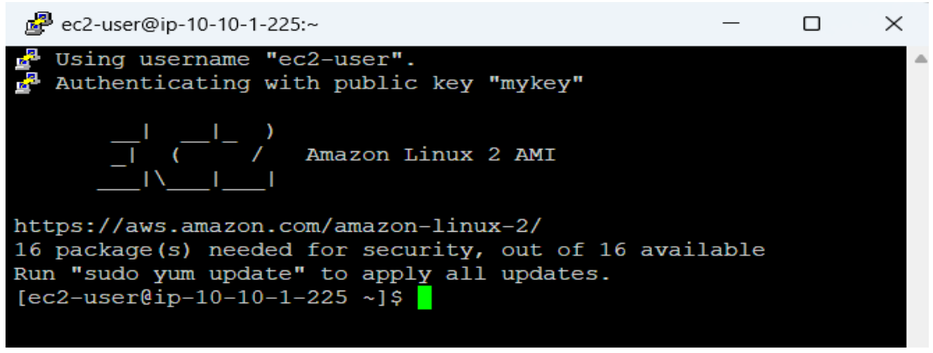
- Entered ec2-user@35.93.143.38 in the **Host Name (or IP address)** field. ec2-user is a default user created in the guest Linux OS, and 35.93.143.38 is the public IP of the EC2 instance.
- The port will be 22.



- Provide the key pair in the PuTTY Configuration window by going to Connection | SSH | Auth



- Click **Open**, and you will be able to SSH into the Linux instance now and have SSH-ed into the cloud EC2 instance



- Note: Since using a Windows terminal to connect to the remote Linux instance, the key pair format is .ppk.
- If you using a Mac or another Linux-based terminal, you will need to use the .pem format.
- These two formats can be converted using the open source software PuTTYgen, which is part of the PuTTY family.

- With a Linux-based terminal including Mac, use the following command to connect to the cloud Linux EC2 instance:

```
ssh -i keypair.pem ec2-user@35.93.143.38
```

- keypair.pem is the key pair file in .pem format.
- Make sure it's set to the right permission using the chomd 400 keypair.pem
- Linux command. ec2-user@35.93.143.38 is user@ EC2's public IP address.
- The default user may change to ubuntu if the EC2 instance is an Ubuntu Linux distribution.

Benefits of cloudshell
- Faster than Console
- Great for demos
- Easy for cloud admins
- Works directly from browser

Launching EC2 Using AWS CLI (Local Terminal)

- Best method for automation and DevOps.

Steps:
Install AWS CLI → Create IAM User → aws configure → Verify → Launch EC2

1. Download, install AWS CLI v2 from AWS docs, and check version using aws –version
2. Create an IAM User for CLI Access
    - Before configuration, create:
        - IAM User
        - Enable Programmatic Access
        - Attach permissions:
            - AmazonEC2FullAccess *(for learning/demo)*
            - Better: custom least-privilege policy in production
        - It give Access Key ID and Secret Access Key (save it for use)

3. Configure AWS CLI using **aws configure** command.
    - AWS will prompt for 4 values
    - AWS stores these locally in:
        - Linux/macOS: ~/.aws/credentials
        - Windows: C:\Users\<user>\.aws\credentials
4. Verify CLI is Working using **aws sts get-caller-identity** command**.**
    - Output: {
      "UserId": "AIDA...",
      "Account": "123456789012",
      "Arn": "arn:aws:iam::123456789012:user/demo-user"
      }

5. Check Available Key Pairs using **aws ec2 describe-key-pairs** command.

6. Launch EC2 Instance using following command:

```
aws ec2 run-instances \
--image-id ami-0abcdef1234567890 \
--instance-type t2.micro \
--count 1 \
--key-name my-key \
--security-group-ids sg-123456 \
--subnet-id subnet-123456
```

Managing EC2 Instances

➢ Start instance, Stop instance , Reboot , Terminate , Resize instance , Attach EBS volume , Modify security groups , Assign Elastic IP , Create AMI , Monitor with CloudWatch.

Accessing EC2 instances securely

- Secure access to EC2 instances ensures that only authorized users can connect to the server while minimizing exposure to cyber threats.

AWS Systems Manager Session Manager
- Recommended by AWS
- No need to open inbound SSH port **22**
- IAM-based access control
- Session logging and auditing

EC2 Instance Connect
- Uses temporary one-time SSH keys
- Browser-based or CLI access
- Reduces long-term key management risks

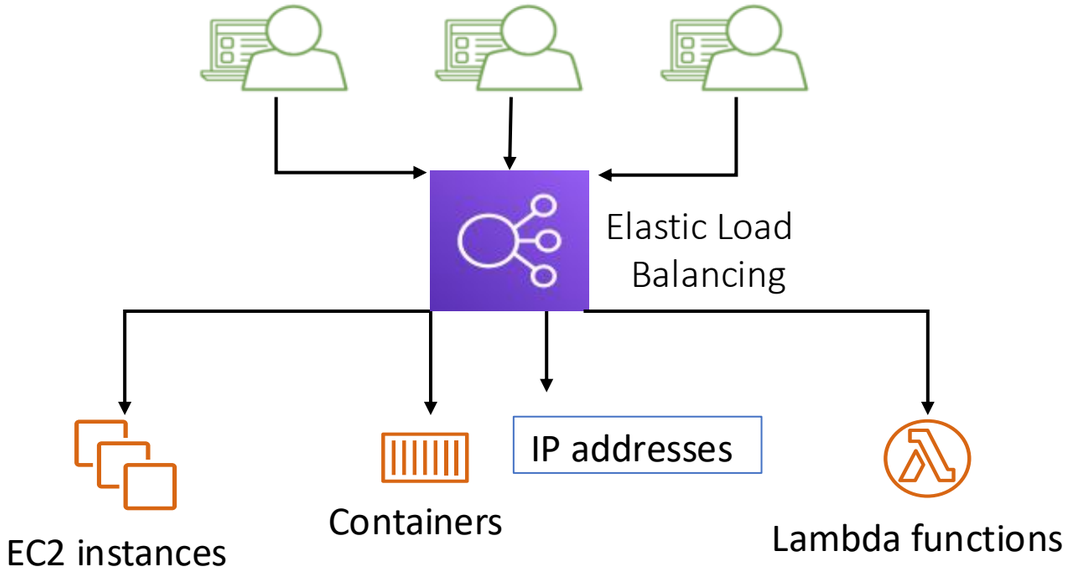
SSH with Key Pairs
- Use .pem private keys
- Disable password login
- Restrict file permission: chmod 400 key.pem
- Best for admin-level Linux access

Security Best Practices
- Allow SSH only from trusted IP addresses
- Avoid 0.0.0.0/0 for port 22
- Use private subnets + bastion host / SSM
- Enable MFA for AWS console users
- Use least privilege IAM roles
- Rotate SSH keys regularly
- Enable CloudTrail + session logs

- The most secure way to access EC2 is by using Session Manager
  Or
- EC2 Instance Connect, as they avoid permanent SSH exposure and provide better auditing.

Elastic Load Balancing (ELB): Traffic Distribution

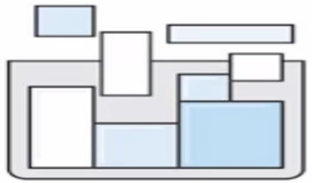
- Distributes incoming application or network traffic across multiple targets in a single Availability Zone or across multiple Availability Zones.
- ELB provides fault tolerance, capacity scaling, and deep integration with Auto Scaling Groups.



Use cases of ELB



Highly available and fault-tolerant applications



Containerized applications



Elasticity and scalability



Virtual private cloud (VPC)



Hybrid environments



Invoke Lambda functions over HTTP(S)

Types of load balancing

Application Load Balancer (ALB)

Layer 7 (HTTP/HTTPS) Load Balancing with advanced routing capabilities. Routing based on path, host headers, HTTP headers, query strings, or source IP addresses. Supports WebSockets, HTTP/2, gRPC, and Lambda targets.

Used for: Microservices, containerized applications, API gateways, web applications

Network Load Balancer (NLB)

Layer 4 (TCP/UDP/TLS) Load Balancing with extremely high performance—millions of requests per second with ultra-low latency. Preserves source IP addresses and supports static IP addresses or Elastic IPs.

Used for: Gaming servers, IoT, real-time communication, ultra-high performance requirements

Gateway Load Balancer (GWLB)

Layer 3 (Network) Load Balancing

Network gateway and transparent load balancer for deploying, scaling, and managing third-party virtual appliances such as firewalls, intrusion detection, and deep packet inspection.

Used for: Inline security appliances, transparent traffic inspection

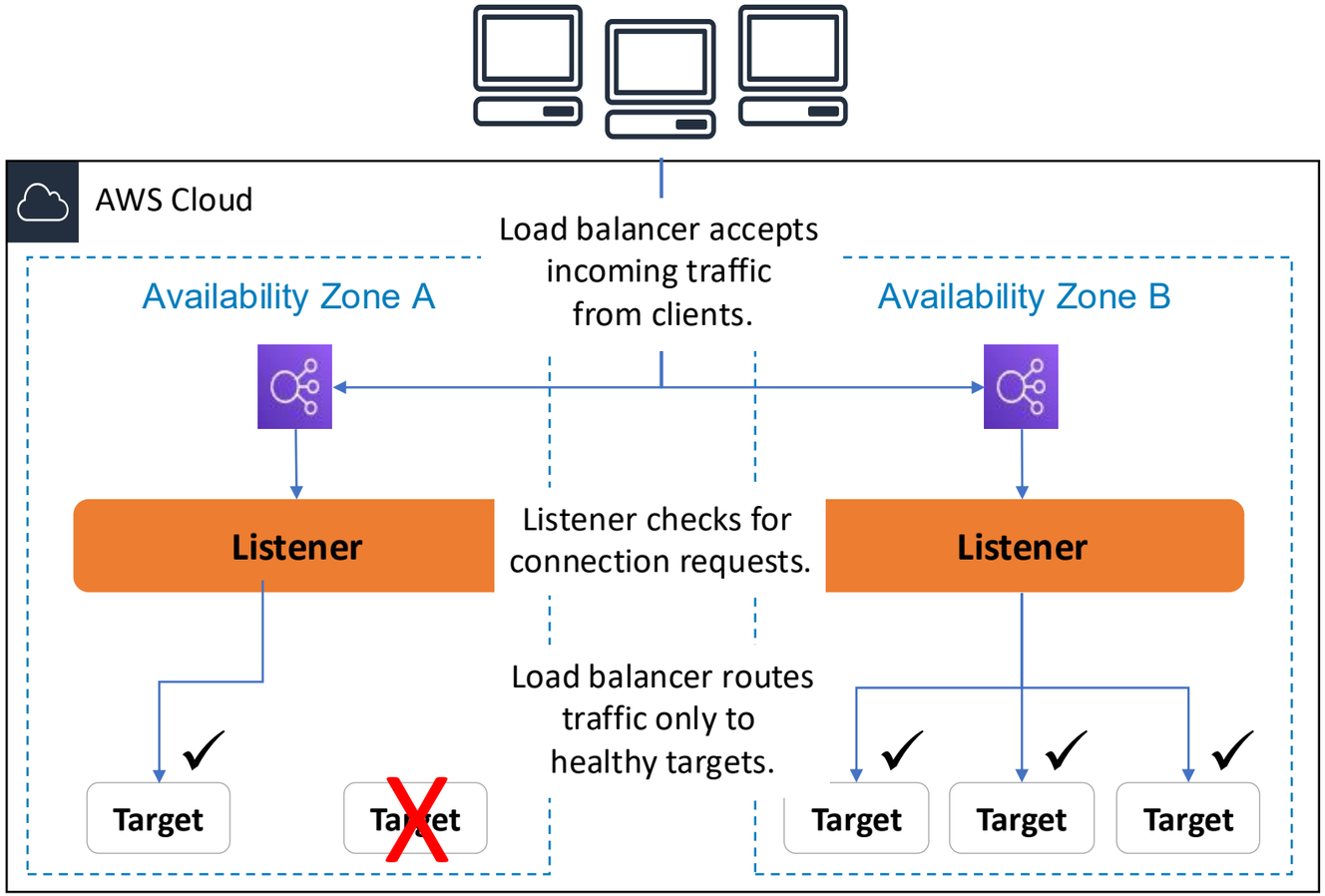
- With Application Load Balancers, Network Load Balancers, and Gateway Load Balancers, you register targets in target groups, and route traffic to the target groups.

Target Groups

- ALB forwards requests to registered targets in a target group. A target group can contain EC2 instances, IP addresses, Lambda functions, or even other Load Balancers.
- Health Checks: ALB continuously monitors the health of targets and stops routing to unhealthy targets. Configure health check path, interval, threshold, and timeout.
- Sticky Sessions: Enable session stickiness so that requests from the same client are sent to the same target, useful for stateful applications.

Target Types

- **Instance:** Register EC2 instance IDs; traffic will go to the primary private IP
- **IP:** Register specific IP addresses, including on-premises servers via VPN/Direct Connect
- **Lambda:** Trigger Lambda functions to respond to HTTP(S) requests (ALB only)
- **Application Load Balancer:** Chain ALBs for advanced scenarios



Load balancer monitoring

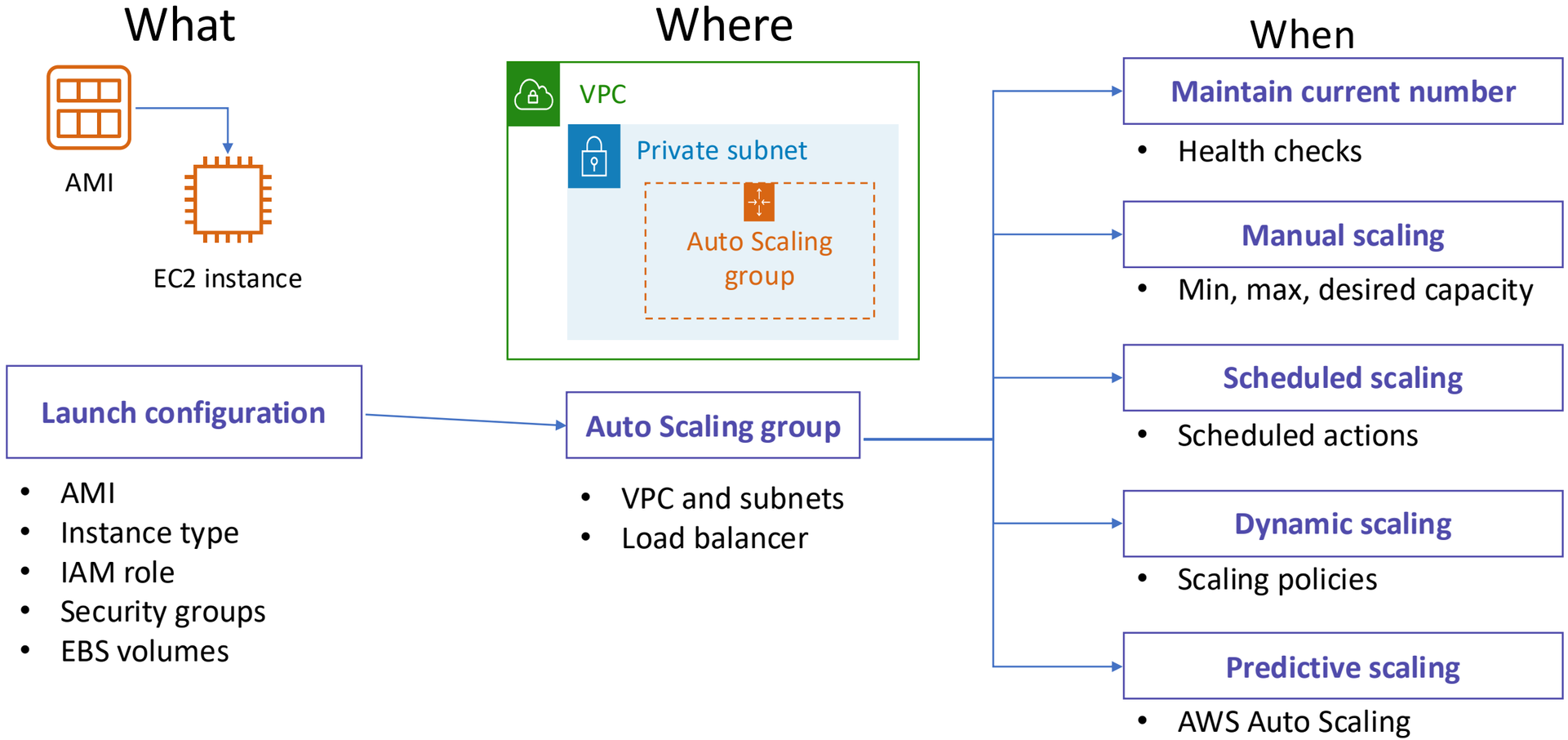
- **Amazon CloudWatch metrics**
- **Access logs:** Capture detailed information about requests sent to your load balancer.
- **AWS CloudTrail logs:** Capture the who, what, when, and where of API interactions in AWS services.

Auto Scaling Group (ASG): Flexible Capacity

- Auto Scaling Groups (ASG) automatically adjust the number of EC2 instances to respond to changes in demand or health status. ASGs are the foundation of elastic architectures that scale out during peak times and scale in during off-peak times to optimize costs.

- **Capacity Management:** Define minimum, desired, and maximum capacity. ASG maintains the desired capacity and operates within defined limits.
- **Health Checks:** Monitor instance and Load Balancer health. Automatically replace unhealthy instances.
- **Load Balancer Integration:** Register instances with Target Groups. Traffic is automatically routed to new instances.
- **Multi-AZ Deployment:** Distribute instances across multiple Availability Zones to ensure fault tolerance and high availability.

How Amazon EC2 Auto Scaling works



Auto Scaling Policies

01

Target Tracking (Recommended)

Specify a target value for a CloudWatch metric such as average CPU utilization or number of requests per target. ASG will automatically create and adjust CloudWatch alarms to maintain your target.

Example: "Maintain average CPU utilization at 50%"

02

Step Scaling

Define scaling actions that vary based on the alarm breach magnitude. Scale out by 2 instances if CPU > 70%, scale out by 4 instances if CPU > 85%.

03

Simple Scaling

An older policy type that adds/removes a fixed number of instances based on a single CloudWatch alarm. Step Scaling or Target Tracking should be used instead.

04

Scheduled Scaling

Scale based on predictable time patterns. Increase capacity before business hours, decrease after business hours. Perfect for fixed-schedule batch jobs.

05

Predictive Scaling

Uses machine learning to analyze historical CloudWatch data and forecast future demand. Automatically schedules scaling actions ahead of predicted load changes.

Pros & cons comparison of EC2 vs Containers vs Serverless:

Model	Pros	Cons
EC2 (VMs)	Full OS control, easy for legacy apps, supports long-running workloads, custom networking/storage	Need to manage OS, patching, scaling, higher ops effort.
Containers (ECS/EKS/Docker)	Portable, lightweight, fast deployment, great for microservices, better resource utilization	Requires orchestration (ECS/K8s), monitoring complexity, learning curve.
Serverless (Lambda)	No server management, auto scaling, pay-per-use, best for event-driven apps	Cold starts, execution time limits, vendor lock-in, less runtime control

EC2 = maximum control
Containers = portability + scalability
Serverless = zero management + pay only when used

Why need ELB and ASG is important

- ELB balances traffic, while ASG ensures the right number of servers are always running.